

LLD — Lead Backend / API Engineer (Next.js API routes)

Overview

This document describes the Low-Level Design (LLD) for the Lead Backend / API Engineer role deliverables shown in your screenshot:

- Next.js API routes: `/api/movies/*`
- `/api/chat`
- TMDB API client integration
- `/api/hackathon/recommend` cache gateway route
- Consistent API error handling

Note: If you share the missing requirement pages later, I can tailor the endpoints, schemas, and flows to match them exactly.

1) Goals & Non-Goals

Goals

- Provide movie discovery/detail endpoints backed by TMDB.
- Provide a chat endpoint for conversational queries/recommendations.
- Provide a recommendation gateway route with caching to reduce latency and upstream calls.
- Standardize validation, logging, and error responses across all routes.

Non-Goals

- Frontend/UI screens.
- Authentication/authorization product decisions (unless required later).

2) Proposed API Surface (v1)

2.1 Movies

1. `GET /api/movies/search?q={query}&page={n}`
2. `GET /api/movies/trending?timeWindow=day|week`
3. `GET /api/movies/{id}`
4. (Optional) `GET /api/movies/{id}/similar`

2.2 Chat

1. `POST /api/chat`

2.3 Recommendation Gateway

1. `POST /api/hackathon/recommend`
-

3) Architecture & Folder Structure

3.1 Layering

Route Handler → **Validation/Auth** → **Service** → **Clients (TMDB/LLM/Cache)** → **Response**

- **Route handler:** HTTP parsing, validation, status codes
- **Service layer:** business logic and composition
- **Clients:** encapsulate external systems (TMDB, LLM, Redis)
- **Shared utilities:** error mapping, request-id, logging

3.2 Suggested structure

If using **App Router**:

```
src/  
  app/api/  
    movies/search/route.ts  
    movies/trending/route.ts
```

```
movies/[id]/route.ts
chat/route.ts
hackathon/recommend/route.ts

server/
  services/
    movies.service.ts
    chat.service.ts
    recommend.service.ts
  clients/
    tmdb.client.ts
    llm.client.ts
    cache.client.ts
  schemas/
    movies.schema.ts
    chat.schema.ts
    recommend.schema.ts
  utils/
    errors.ts
    logger.ts
    requestId.ts
    env.ts
    http.ts
    rateLimit.ts
```

4) Data Contracts (Request/Response)

4.1 GET /api/movies/search

Query

- `q` (required, 2–100 chars)
- `page` (optional, default 1)

Response (200)

```
{
  "results": [
    {
      "id": 603,
      "title": "The Matrix",
      "overview": "...",
      "posterUrl": "https://image.tmdb.org/t/p/w500/...jpg",
      "releaseDate": "1999-03-30",
      "rating": 8.2
    }
  ],
  "page": 1,
  "totalPages": 20
}
```

4.2 GET /api/movies/trending

Query

- `timeWindow` (optional) `"day" | "week"` default `"week"`

Response (200)

Same shape as search but without `page/totalPages` if you want simpler output.

4.3 GET /api/movies/{id}

Path

- `id` number

Response (200)

```
{
  "id": 603,
  "title": "The Matrix",
  "overview": "...",
  "genres": ["Action", "Sci-Fi"],
  "runtimeMins": 136,
}
```

```
"releaseDate": "1999-03-30",
"posterUrl": "...",
"backdropUrl": "...",
"rating": 8.2
}
```

4.4 **POST** /api/chat

Request

```
{
  "messages": [
    { "role": "user", "content": "Recommend action movies like John Wick" }
  ],
  "context": {
    "userId": "optional",
    "locale": "en-IN"
  }
}
```

Response (200)

```
{
  "reply": { "role": "assistant", "content": "..." },
  "metadata": { "requestId": "...", "latencyMs": 1200 }
}
```

4.5 **POST** /api/hackathon/recommend

Request

```
{
  "seed": {
    "movieIds": [603, 245891],
    "genres": ["Action"]
  }
}
```

```
  },  
  "limit": 10  
}
```

Response (200)

```
{  
  "recommendations": [  
    { "id": 155, "title": "The Dark Knight", "reason": "..."  
  }  
  ],  
  "metadata": { "cache": "hit", "requestId": "..."  
}
```

5) TMDB Client (Integration)

5.1 Configuration

Environment variables:

- `TMDB_API_KEY`
- `TMDB_BASE_URL` (default `https://api.themoviedb.org/3`)

5.2 Client responsibilities

- Build request URL and attach the TMDB key consistently.
- Centralize timeouts, retries, and response parsing.
- Normalize TMDB errors into typed internal errors that the API layer can map to consistent HTTP responses.
- Normalize TMDB responses into a stable internal model (so the frontend is not coupled to TMDB response shapes).

5.3 Authentication pattern (TMDB)

Use one approach consistently (pick based on your TMDB plan):

- **Bearer token** (preferred): send `Authorization: Bearer <TMDB_READ_TOKEN>` \n or
- **API key query param**: append `?api_key=...`

Do not mix both approaches across endpoints.

5.4 HTTP client conventions

Standardize request behavior in one place:

- timeout: 8–12s per request (fail with upstream timeout)
- retries: only for **idempotent** GET requests and only for transient failures (network/5xx)
- always include `Accept: application/json`
- propagate `x-request-id` to upstream calls (as a header) for debugging

5.5 Functions (exact signatures)

In `server/clients/tmdb.client.ts` expose:

- `searchMovies({ query, page }): Promise<TmdbSearchResult>`
- `getTrending({ timeWindow }): Promise<TmdbTrendingResult>`
- `getMovieDetails({ id }): Promise<TmdbMovieDetails>`
- `getSimilarMovies({ id, page? }): Promise<TmdbSimilarResult>`

And in `server/services/movies.service.ts` expose normalized outputs:

- `searchMovies({ q, page }): Promise<MovieListResponse>`
- `getTrending({ timeWindow }): Promise<MovieListResponse>`
- `getMovie({ id }): Promise<MovieDetailsResponse>`
- `getSimilar({ id, page? }): Promise<MovieListResponse>`

5.6 Normalization rules (important)

Normalize into a consistent internal “Movie” object:

- `posterUrl` and `backdropUrl`: build using TMDB image base URL (e.g. `https://image.tmdb.org/t/p/w500`)

- `rating` : map from `vote_average`
- `releaseDate` : map from `release_date`
- handle missing/null fields safely (return `null` or omit as per your API contract)

5.3 Key functions

- `searchMovies(query: string, page: number)`
- `getTrending(timeWindow: "day" | "week")`
- `getMovieDetails(id: number)`
- `getSimilarMovies(id: number)`

6) LLM/Chat Integration (for `/api/chat`)

6.1 Responsibilities

- Validate user input (messages, size limits).
- Apply safe defaults: max tokens, temperature, model name.
- Optionally ground chat with movie context by calling `MoviesService` (search/title resolution).
- Produce a stable response format for the frontend.

6.2 Environment variables (example)

- `LLM_PROVIDER` (e.g., `openai` / `azure` / `local`)
- `LLM_API_KEY`
- `LLM_MODEL` (e.g., `gpt-4.1-mini` or as per your stack)
- `LLM_BASE_URL` (optional; required for some providers)

6.3 Client shape

In `server/clients/llm.client.ts` implement:

- `generateChatCompletion({ messages, systemPrompt, model, temperature }): Promise<{ content: string }>`

6.4 Guardrails & limits

- Max messages: e.g., 20
 - Max content length per message: e.g., 2–4k chars
 - PII/secrets: never log full prompts in production logs
 - Timeout: 15–25s (LLM calls can be slower than TMDB)
-

7) Cache Gateway (Redis recommended)

6.1 Cache-aside strategy

1. Compute a deterministic key
2. `GET key` → if present return
3. Else compute → `SET key value EX ttl`

6.2 Suggested cache keys + TTL

- `movies:search:{q}:{page}` → 10–30 min
- `movies:trending:{timeWindow}` → 30–60 min
- `movies:details:{id}` → 6–24 hrs
- `reco:v1:{hash(seed)},{limit}` → 15–60 min

6.3 What gets cached

- Final normalized API payloads (fastest)
 - Optionally cache raw TMDB payloads separately if you need flexibility
-

8) API Design Details (per route)

8.1 Common headers

- Request: `x-request-id` (generated if missing)
- Response: `x-request-id`

8.2 GET /api/movies/search

Integration calls:

1. Cache lookup: `movies:search:{q}:{page}` \n
2. On miss: `TMDB /search/movie` \n
3. Normalize response and cache it

8.3 GET /api/movies/{id}

Integration calls:

1. Cache lookup: `movies:details:{id}` \n
2. On miss: `TMDB /movie/{id}` (optionally add `append_to_response=credits,videos`) \n
3. Normalize response and cache it

8.4 POST /api/chat

Integration calls:

1. Validate messages \n
2. (Optional) call `MoviesService.searchMovies` to ground context \n
3. Call `LLMClient.generateChatCompletion` \n
4. Return assistant message + metadata

8.5 POST /api/hackathon/recommend (cache gateway)

Integration calls:

1. Compute deterministic cache key \n
 2. Cache lookup \n
 3. On miss: compute recommendations \n
 - Option A (fast): TMDB similar + ranking \n
 - Option B (hybrid): TMDB candidate set + LLM re-rank \n
 4. Cache response + return
-

9) Validation, Error Handling, and Response Standards

7.1 Validation

Use `zod` schemas per route:

- query params for GET
- body for POST

7.2 Standard error format

All errors return:

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "q is required",
    "requestId": "..."
  }
}
```

7.3 Error mapping table

- Zod validation error → **400**
- Not found (movie id) → **404**
- Rate limited (TMDB 429) → **429**
- Upstream down/timeouts → **503**
- Unknown → **500**

10) Observability

- Add `requestId` to every request/response (header `x-request-id`)
- Structured logs per route:
 - route, method, status, latency, requestId, upstream latency

- Avoid logging secrets and full user prompts if not needed
-

11) Rate Limits & Timeouts

- `/api/chat` : strict rate limit by IP/user (example: 10 req/min)
 - Upstream timeout: 8–12 seconds
 - Fail fast with clear 503 on upstream failure
-

12) Testing Plan

Unit tests

- `tmdb.client` URL construction + error handling
- `recommend.service` cache key determinism + ranking logic

Integration tests

- API handlers return correct status code and schema
 - Cache hit/miss behavior for recommend route
-

13) Your Role (Lead Backend / API Engineer) — Complete Scope

13.1 What you own end-to-end

- Designing the backend API surface (`/api/movies/*` , `/api/chat` , `/api/hackathon/recommend`).
- Implementing integration clients (TMDB, LLM, cache) with clean boundaries.
- Ensuring reliability: timeouts, retries, rate limits, caching.
- Standardizing API contracts: request validation + response/error formats.
- Observability and debuggability: request-id, structured logs, metrics (if available).

13.2 Deliverables (what reviewers expect)

1. Working API routes with documented contracts (request/response examples).\n
2. Integration layer (TMDB + LLM) that is reusable/testable.\n
3. Cache gateway implementation and cache key strategy.\n
4. Error handling and consistent API responses.\n
5. Basic tests (unit + integration) and a short README on how to run.\n

13.3 Definition of Done (DoD)

- All endpoints return stable schemas and correct HTTP status codes.
 - Cache hit reduces latency for trending/details/recommend.
 - Upstream failures return meaningful 4xx/5xx errors (no raw stack traces).
 - Logs include requestId and upstream latency.
 - Secrets stored only in environment variables (never committed).
-

14) Open Questions (to finalize once you share the requirement pages)

- Exact list of `/api/movies/*` endpoints required
- Whether `/api/chat` must store conversation history
- Required recommendation strategy (rules vs LLM vs hybrid)
- Whether auth is required (API key/JWT)
- Target deployment environment (Vercel/server) and whether Redis is available